Capture Module v2 — Technical Design

Sentiora — Group 04, MIT School of Computing, MIT-ADT University Architecture + Security Lead: Vansh Asnani

1. Core Goal

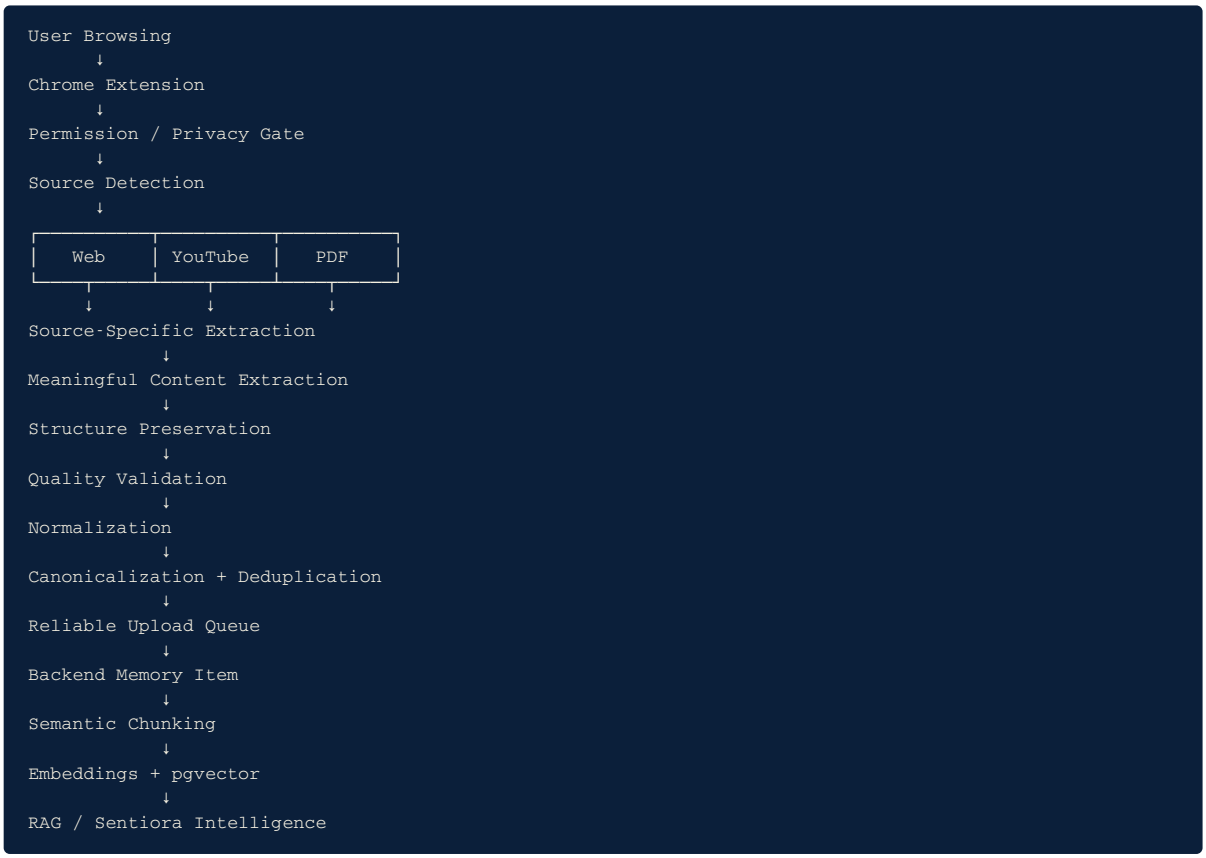
Sentiora's Capture Module currently captures basic webpage metadata — title, URL, and thumbnail — which is insufficient for downstream intelligence. Capture Module v2 must instead preserve the **meaningful content a user actually viewed and engaged with**, structured well enough for the Intelligence/RAG layer to answer questions grounded in it.

Example: A user searches "Binary Search," opens GeeksForGeeks, reads the explanation, algorithm, examples, and code, then clicks **Capture Memory**. Sentiora must store that content — not just the page title and link — so it can later answer *"What did I learn about Binary Search?"*

Current State → Target State

	Current State	Target State (v2)
Captured data	Title, URL, thumbnail	Full meaningful content, structure, code, metadata
RAG readiness	Not usable for grounded answers	Chunked, embedded, citable
Extraction	Naive/none	Source-specific, quality-validated

2. Target Architecture



3. Capture Requirements

Source	Capture
Webpage	Main content, headings, paragraphs, lists, tables, code, links, metadata
YouTube	Transcript, title, channel, description, thumbnail, transcript language
PDF	Full text, document structure, metadata; robust handling of large PDFs
Manual Capture	Explicit user-triggered capture of the current meaningful content
Automatic Capture	Conservative, privacy-controlled capture

---

4. Open-Source Strategy

Sentiora should **reuse proven open-source extraction technology instead of rebuilding everything from scratch**, while keeping a Sentiora-owned abstraction layer around every dependency.

Potential components:

- **Mozilla Readability** — primary webpage article extraction
- **Trafilatura** — backend/fallback web extraction where useful
- **PDF.js / suitable PDF parser** — robust PDF extraction
- **Unstructured** — evaluate for structured document/PDF extraction

> ***Rule:*** *Do not add dependencies blindly. Cursor must inspect the existing implementation and integrate an open-source component only when it materially improves capture quality.*

```
Open-Source Libraries
        ↓
Sentiora Adapters
        ↓
Sentiora Capture Contract
        ↓
Memory Pipeline
```

---

5. Capture Data Contract

```
Memory Capture
├── source metadata
├── canonical URL
├── meaningful content
├── headings / sections
├── code / lists / tables
├── author / site / dates
├── capture method
├── extraction method
├── extraction quality
├── content fingerprint
└── captured_at
```

Metadata alone is NOT a successful meaningful capture.

---

6. Reliability & Privacy

- Domain blocklist
- Sensitive-page detection
- Password/login protection
- Manual vs. automatic capture
- Local persistent upload queue
- Retry with backoff
- Duplicate prevention
- Extraction failure states
- No fabricated content when extraction fails

7. E2E Definition of Done

```
Browse Page
    ↓
Capture
    ↓
Meaningful Content Stored
    ↓
Memory Visible in Dashboard
    ↓
Content Chunked + Embedded
    ↓
Semantic Search Finds It
    ↓
Ask Sentiora
    ↓
Grounded Answer + Citation
```

Capture Module v2 is complete only when this end-to-end flow works reliably.

---

8. Implementation Principle

1. Inspect existing Sentiora code first.
2. Reuse existing working components where possible.
3. Integrate open-source libraries through adapters, not direct coupling.
4. Do not add unnecessary dependencies.
5. Preserve meaningful structure, not just plain text.
6. Never treat title/URL/thumbnail-only data as successful meaningful capture.
7. Every change must include tests.
8. Run lint, typecheck, tests, and build before completion.